

Lesson 2 — A Game Is Data

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

A struct bundles related variables

```
struct Fighter {
    std::string name;
    int health;
    int max_health;
    int attack;
    int defence;
};           // <- the semicolon matters
```

```
Fighter troll;
troll.name = "Cave troll";
troll.health = 120;
```

A third fighter is **one line**.

& means THE ACTUAL ONE

```
void hurt(Fighter f) { f.health -= 10; }
// a PHOTOCOPY - changes vanish
void hurt(Fighter& f) { f.health -= 10; }
// THE REAL ONE - changes stick
```

Getting this wrong gives **no error and no warning**. The program runs and does nothing.

Written	Means	Use when
<code>Fighter f</code>	a copy	read-only and small
<code>Fighter& f</code>	the real one	you want to change it
<code>const Fighter& f</code>	the real one, read-only	read-only and big

Clamping

```
#include <algorithm>
health = std::max(0, health - damage);
health = std::min(max_health, health + heal);
```

Integer division: multiply FIRST

```
int filled = (health * WIDTH) / max_health;
// right
int filled = (health / max_health) * WIDTH;
// ALWAYS 0
```

With whole numbers, `health / max_health` is 0 for anything less than full.

What data costs

```
sizeof(char)    // 1
sizeof(int)     // usually 4
sizeof(double)  // usually 8
sizeof(Fighter) // the whole bundle
```

100,000 particles using `double` vs `float`: **3 MB vs 1.5 MB** — and the smaller one may run twice as fast, because it fits in the cache.

Overflow

An `int` holds about ± 2.1 billion. Add one past the top and it wraps to the bottom.

Section A — Recall

A1. What is a `struct`? Give two reasons to use one rather than loose variables. [3 marks]

A2. Explain what `&` does in `void hurt(Fighter& f)`. What happens if you leave it out? [3 marks]

A3. When would you use `const Fighter&` rather than `Fighter`? [2 marks]

A4. A fighter has 45 health out of 200, and the bar is 16 wide. How many `#` should it show? Give the expression and the answer. [3 marks]

A5. Why might a game choose `float` over `double` for 100,000 particles? Give the reason that is *not* about running out of memory. [2 marks]

Section B — Predict the output

B1. What does this print, and why? [4 marks]

```
void heal(Fighter f) { f.health += 50; }

Fighter troll;
troll.health = 100;
heal(troll);
std::cout << troll.health << "\n";
```

B2. A fighter has 80 health out of 100 and the bar is 16 wide. What does each version print, and why is one of them always empty? [4 marks]

```
int a = (80 * 16) / 100;
int b = (80 / 100) * 16;
```

B3. What is printed?

[3 marks]

```
int score = 2147483647;
std::cout << score << "\n";
std::cout << score + 1 << "\n";
```

B4. Does this compile? If not, what is the error, and is the compiler right to complain?

[4 marks]

```
void describe(const Fighter& f) {
    f.health = 999;
    std::cout << f.name;
}
```

Section C — Find and fix the bug

C1. The battle runs forever. Nobody ever takes any damage. The program compiles with **no errors and no warnings**. What is wrong?

[4 marks]

```
void do_attack(Fighter attacker, Fighter defender) {
    defender.health -= attacker.attack;
}
```

C2. This does not compile, and the error message points at whatever comes after it. What is missing?

[3 marks]

```
struct Fighter {
    std::string name;
    int health;
}
```

C3. A fighter is created and immediately shows a health bar full of nonsense — sometimes enormous, sometimes negative, different every run. Why?

[4 marks]

```
Fighter goblin;
goblin.name = "Goblin";
draw_health_bar(goblin);
```

Section D — Build it

- ☐ **Checkpoint 1.** Write a `Fighter` struct with name, health, max_health, attack and defence. Make one and print its fields.
- ☐ **Checkpoint 2.** Write `make_fighter(name, health, attack, defence)` that returns a filled-in `Fighter` with `max_health` set automatically.
- ☐ **Checkpoint 3.** Write `draw_health_bar(const Fighter&)` using `#` and `-`. Check it is right at full health, half health and zero.
- ☐ **Checkpoint 4.** Write `do_attack(Fighter& attacker, Fighter& defender)`. **Write it without the `&` first**, run it, and watch nobody take damage. Then add the ampersands.
- ☐ **Checkpoint 5.** Build the battle loop: attack, defend, potion and run, with the enemy taking its turn. End the fight when either side reaches 0.
- ☐ **Checkpoint 6.** Add a third fighter that joins the battle. Count how many lines it took.

CHECKPOINT 4 IS NOT A MISTAKE IN THE INSTRUCTIONS

You are being asked to write the bug on purpose. Seeing a program compile cleanly, run perfectly, and do absolutely nothing is the most valuable thirty seconds in this lesson. You will meet this bug for real later, and you will recognise it.

Section E — Design challenge

NO RIGHT ANSWER

E1. Design the data for an **item** — a sword, a potion, a key — so that all of them can live in the same inventory. What fields does `Item` need? What goes wrong if you give it every field that any item might ever need?

E2. Your `Fighter` holds a `std::string`. Print `sizeof(Fighter)`, then make the name very much longer and print it again. The number does not change. **Work out why.** What is actually stored inside the struct?

E3. Copies are safe and slow. References are fast, and let a function change your data behind your back. Python makes this choice for you and never mentions it. Which do you prefer, and what does C++'s way let you do that Python's does not?

Stretch goals

1. A third fighter. How many lines?
2. An `Item` struct and a small inventory.
3. Critical hits — a small chance of double damage, with a message so the player knows.
4. A `std::vector<Fighter>` and a loop taking everyone's turn. (That is lesson 5 arriving early.)
5. **Measure it.** Time a million calls by value against a million by `const&`. Then make the struct much bigger and try again.